

Cahier de conception

“PROJET DE PROGRAMMATION ”

Sujet

Développement d'un système de recommandation basique

Réalisé par :

Dihya MEGHERFI
Dina BOUDERKA
Lilia JESOVER-ENGEL
Adriana NGANZU NDAYIDI

Encadré par :

M Enzo BONNEAU

Année : 2025/2026

Table des matières

1	Introduction	4
1.1	Objectifs et méthodes	4
1.2	Documents de référence	4
2	Guide de lecture	5
2.1	Maîtrise d'œuvre	5
2.2	Maîtrise d'ouvrage	6
3	Concepts de base	7
4	Conception générale	8
4.1	Grands domaines fonctionnels	8
4.2	Architecture de l'application	9
4.2.1	Paradigme de programmation utilisé	9
4.2.2	Principaux composants logiciels	9
4.3	Diagrammes UML	10
5	Conception détaillée	13
5.1	Description des modules — Frontend	13
5.1.1	Interface utilisateur – application Vue.js	13
5.1.2	Module de navigation - Vue Router	14
5.1.3	Module Composants Vue	15
5.1.4	Module Communication API	15
5.2	Description des modules — Backend	16
5.2.1	Module Authentification — AWS Cognito (Amplify Auth)	16
5.2.2	Module Schéma de données — Amazon DynamoDB	17
5.2.3	Module Couche API — AppSync (GraphQL) + API Gateway (REST)	20
5.2.4	Module Lambda Python — Algorithme de recommandation	22
5.2.5	Module Amazon S3 — Stockage des fichiers	26
6	Annexes	27
	Glossaire	28
	Références	31
	Index	32

Liste des tableaux

5.1	Les routes principales de l'application Vue.js	14
5.2	Les composants principaux de l'interface utilisateur	15
5.3	Les services utilisés pour la communication avec les API	16
5.4	Attributs utilisateur générés par Amazon Cognito	16
5.5	Schéma de la collection Movie	17
5.6	Schéma de la collection Rating	18
5.7	Schéma de la collection Preference	18
5.8	Schéma de la collection UserProfile	19
5.9	Opérations exposées par la couche API	21
5.10	Structure des fichiers du module de recommandation	22
5.11	Fonctions d'accès à DynamoDB	23
5.12	Fonctions de l'algorithme de recommandation	24

Table des figures

4.1	Architecture	9
4.2	Diagramme de cas d'utilisation	10
4.3	Diagramme de séquences de la demande de recommandation	11
4.4	Diagramme de séquences de la notation	11
4.5	Diagramme de séquences de la Remise à zéros des préférences	12
4.6	Storyboard	12

Chapitre 1

Introduction

Le livrable de conception détaillée s'inscrit dans le cadre du projet de développement d'un système de recommandation de films. Ce document présente l'architecture prévue du système, la description des principaux composants et modules, ainsi que la structure des données et des fonctionnalités attendues. Il a pour but de servir de référence avant le développement, afin de guider l'implémentation future et d'assurer la cohérence avec le cahier des charges.

La démarche adoptée repose sur une analyse des spécifications fonctionnelles du projet. À partir de ces exigences, les différentes parties du système sont décrites afin de préciser leur rôle et leur fonctionnement au sein de l'application. Cette étape permet de structurer le développement futur et d'assurer une cohérence entre les besoins identifiés et la solution proposée.

1.1 Objectifs et méthodes

L'objectif principal de ce document est de détailler la conception du système de recommandation de films avant sa phase de développement. Il vise à présenter l'organisation générale de l'application, les différents composants du système ainsi que les fonctionnalités prévues pour répondre aux besoins définis dans le cahier des charges.

1.2 Documents de référence

Les documents ayant servi de base à l'élaboration du cahier de conception sont :

- Le cahier des charges du projet.
- Le cahier de recette du projet.
- Les maquettes de l'interface utilisateur.
- La documentation du jeu de données utilisé : **The Movies Dataset**.
- La documentation de l'outil d'hébergement : **AWS**.

Chapitre 2

Guide de lecture

Ce document présente la conception détaillée du système de recommandation de films. Il est destiné à plusieurs types de lecteurs.

2.1 Maîtrise d'œuvre

La maîtrise d'œuvre est constituée des quatre étudiantes responsables de la conception et du développement du système. Le rôle de chef de projet est assuré à tour de rôle par les membres de l'équipe.

- Dihya MEGHERFI
- Dina BOUDERKA
- Lilia JESOVER-ENGEL
- Adriana NGANZU NDAYIDI

Responsable

La personne assurant le rôle de responsable organise les réunions de suivi, coordonne les activités de l'équipe, répartit les tâches et veille au respect du planning ainsi qu'à l'avancement global du projet.

Personnel technique

L'ensemble des membres de l'équipe participe aux activités techniques du projet, notamment :

- Conception de l'architecture cloud AWS (Amplify, Cognito, DynamoDB, AppSync, Lambda)
- Développement du backend Python (Lambda de recommandation)
- Développement du frontend en Vue.js
- Traitement du dataset de films et implémentation de l'algorithme de recommandation
- Réalisation des tests et intégration du système
- Rédaction de la documentation technique

2.2 Maîtrise d'ouvrage

La maîtrise d'ouvrage est représentée par l'UFR Mathématiques et Informatique de l'Université Paris Cité, ainsi que par l'encadrant pédagogique, M. Enzo Bonneau.

La maîtrise d'ouvrage est chargée de définir les objectifs du projet, de valider les fonctionnalités développées, d'évaluer la conformité du système au cahier des charges et d'approuver le produit final.

Chapitre 3

Concepts de base

- **Architecture** : L'architecture d'un système informatique désigne l'organisation générale de ses composants logiciels et matériels ainsi que les interactions entre eux. Elle décrit comment les différentes parties du système sont structurées et collaborent pour assurer le fonctionnement global de l'application.
- **Interface web** : Interface accessible via un navigateur permettant à l'utilisateur d'interagir avec le système. Le système inclut une interface permettant :
 - De consulter les recommandations de films générées.
 - De noter des œuvres.
 - De rechercher des films.

Les tests vérifieront la cohérence entre les actions de l'utilisateur et les résultats affichés.

- **Système de recommandation** : Un système de recommandation est un outil informatique permettant de proposer des éléments à un utilisateur en prédisant ses préférences, en se basant sur les notes qu'il attribue à d'autres éléments ainsi que les informations associées à ces derniers.
- **Jeu de données (dataset)** : Le modèle s'appuie sur un jeu de données qui est un ensemble structuré d'informations. Dans ce projet, il contient des films avec leurs caractéristiques (titre, genre, note moyenne, réalisateur, année de sortie) ainsi que les notes attribuées par des utilisateurs.
- **Base de données** : Une base de données est un ensemble organisé de données stockées sous forme de plusieurs jeux de données.
- **Authentification** : Une méthode chiffrée qui permet aux appareils et aux personnes de s'identifier auprès d'autres appareils et systèmes.

Chapitre 4

Conception générale

Ce chapitre a pour but de présenter la structure générale de l'application Recomodo. On y trouvera ses principales fonctionnalités, son architecture et certaines interactions entre les différents services.

4.1 Grands domaines fonctionnels

L'application Recomodo se divise en plusieurs domaines fonctionnels :

Domaine Fonctionnel	Description
Interface web	Affiche les informations, récupère les actions de l'utilisateur
Connexion et préférences utilisateur	Inscription, connexion, gestion de profil
Business Logic/Middleware	Coordonne les échanges entre les différents services
Système de recommandation	Propose une liste de films recommandés pertinente
Base de données	Données utilisateurs et données des films

4.2 Architecture de l'application

4.2.1 Paradigme de programmation utilisé

L'application repose sur le paradigme de la programmation impérative. Le backend sera constitué de micro-services.

4.2.2 Principaux composants logiciels

- **AWS Amplify** : AWS Amplify est une plateforme de développement qui facilite le déploiement et l'intégration d'applications web avec les services AWS. Dans ce projet, elle est utilisée pour héberger l'interface web et simplifier la connexion avec les services backend.
- **AWS Cognito** : AWS Cognito est un service d'authentification et de gestion des utilisateurs. Il permet de gérer l'inscription, la connexion et la vérification des utilisateurs de manière sécurisée.
- **API Gateway** : L'API Gateway constitue l'interface de communication entre l'application web et les services backend. Elle permet de transmettre les requêtes envoyées par l'interface utilisateur aux bons services dans le backend et de renvoyer les réponses correspondantes.
- **AWS Lambda** : AWS Lambda est un service de calcul sans serveur permettant d'exécuter du code en réponse à des événements. Dans ce projet, les fonctions Lambda implémentent la logique métier et certaines parties du système de recommandation.
- **DynamoDB** : DynamoDB est une base noSQL de données proposée par AWS. Elle nous servira à stocker les informations utilisateur, ainsi que les informations sur les films.
- **AWS S3** : AWS S3 est un service de stockage d'objets utilisé pour conserver les affiches des films, des documents ou d'autres ressources associées aux éléments recommandés.
- **Python** : Le système de recommandation ainsi que d'autres parties du backend seront implémentées en python. Pour le système de recommandation les bibliothèques principales sont **Pandas** et **Sickit-learn**.
- **Vue** : Vue est un framework **javaScript** que nous allons utiliser pour développer l'interface utilisateur interactive et dynamique. Nous utiliserons l'extension **TypeScript** de **JavaScript**.
- **Node.js** : Node.js est un environnement d'exécution **JavaScript** côté serveur. Il permet d'utiliser le même langage pour le frontend et le backend. Nous utiliserons l'extension de **JavaScript** : **TypeScript** pour ce projet.

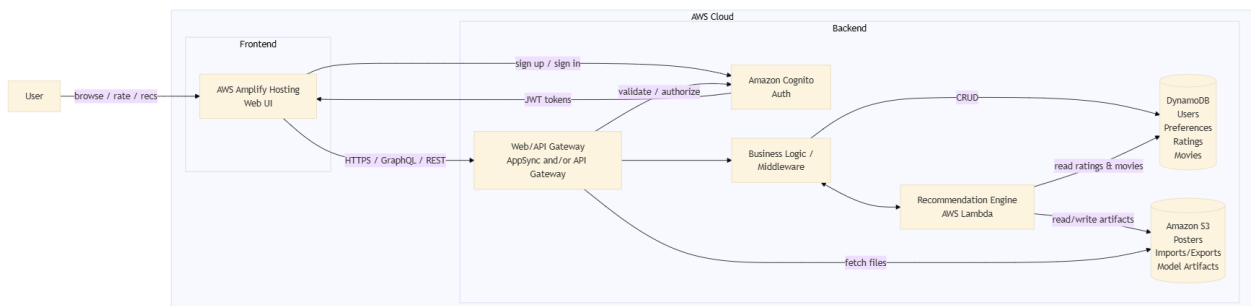


FIGURE 4.1 – Architecture

4.3 Diagrammes UML

Les différentes interactions entre les composants sont modélisées à l'aide des diagrammes UML :

- Diagramme de cas d'utilisation :

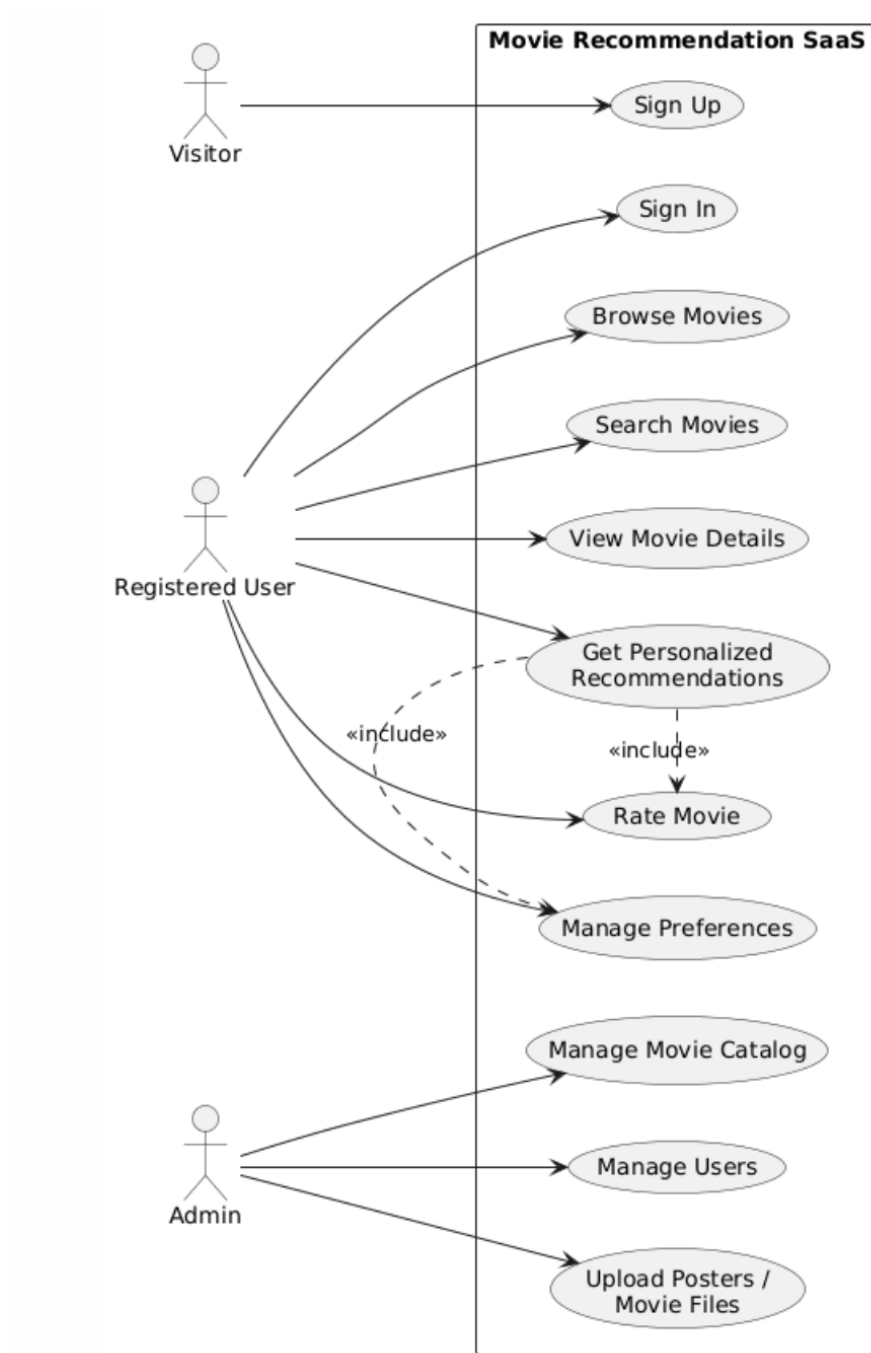


FIGURE 4.2 – Diagramme de cas d'utilisation

— Diagrammes de séquences

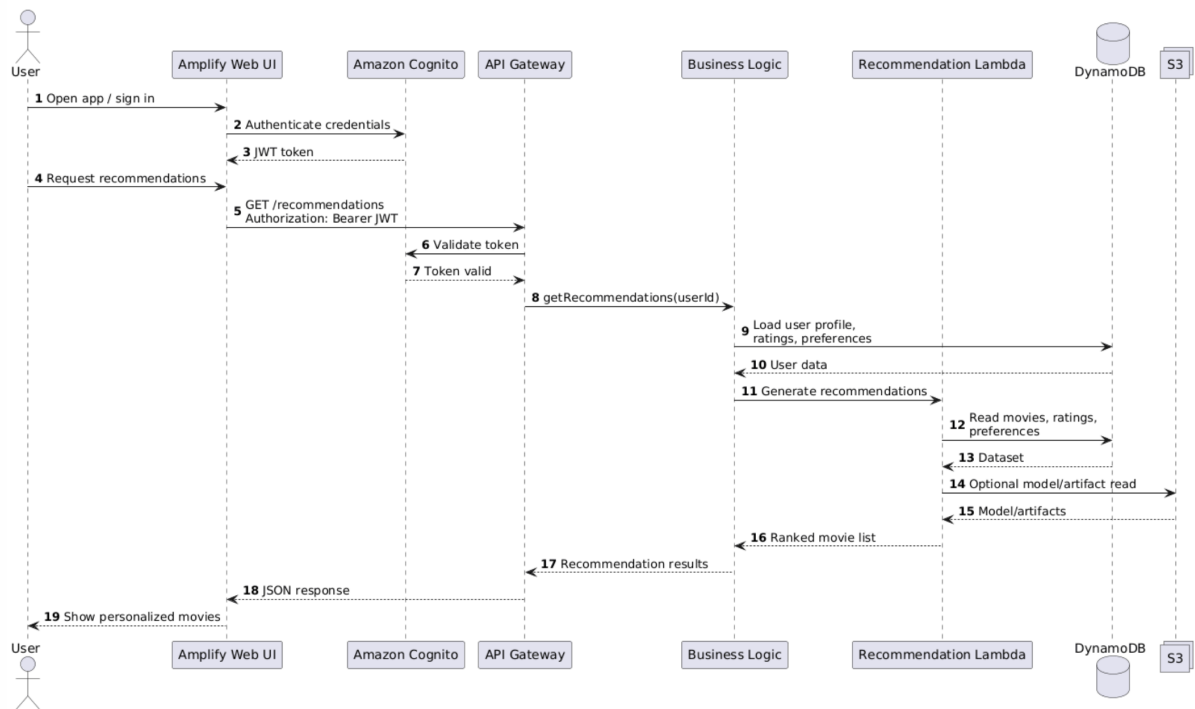


FIGURE 4.3 – Diagramme de séquences de la demande de recommandation

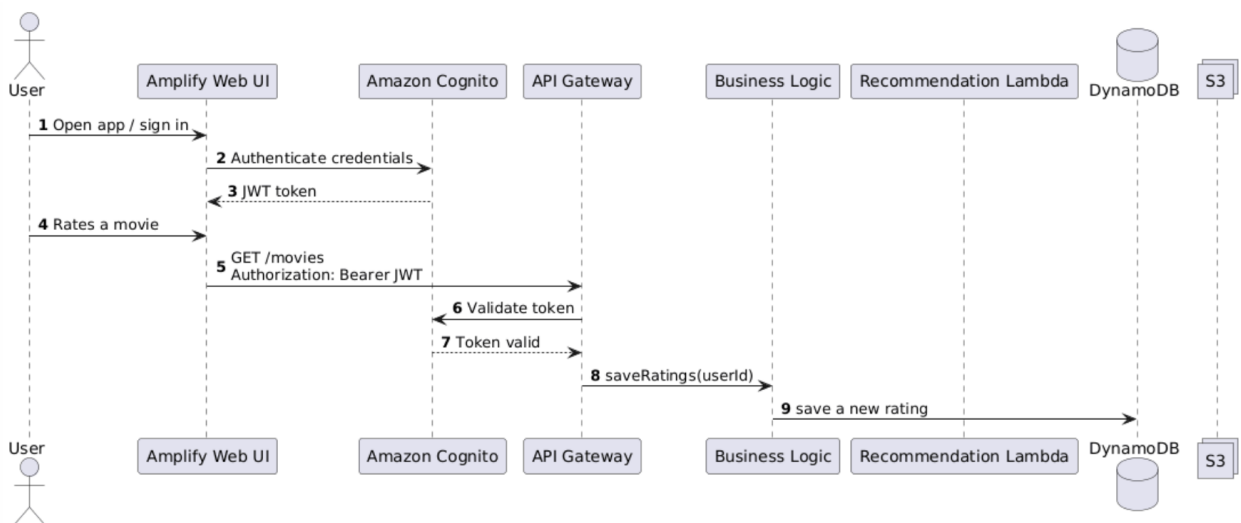


FIGURE 4.4 – Diagramme de séquences de la notation

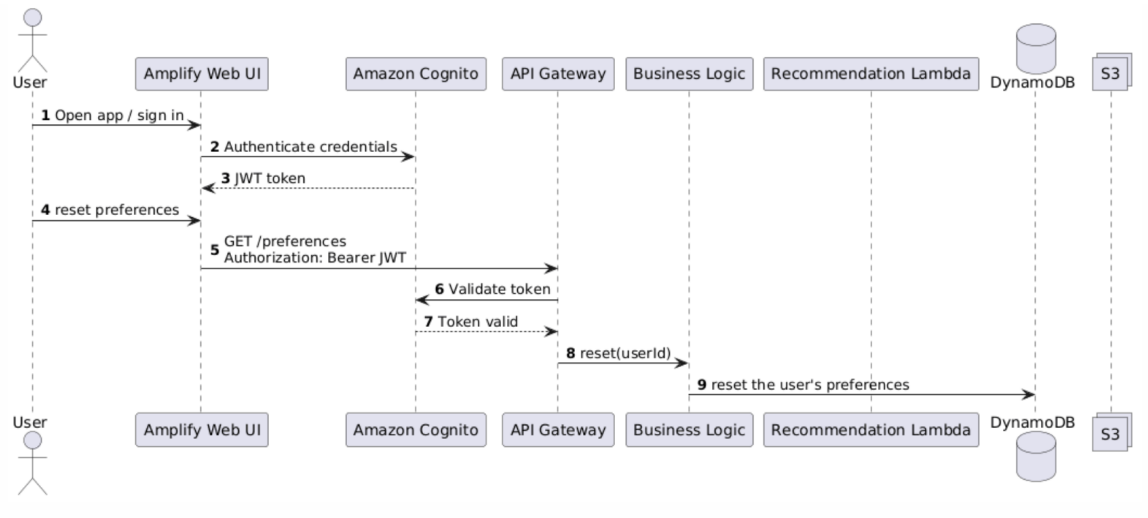


FIGURE 4.5 – Diagramme de séquences de la Remise à zéros des préférences

— "Storyboard" du site web

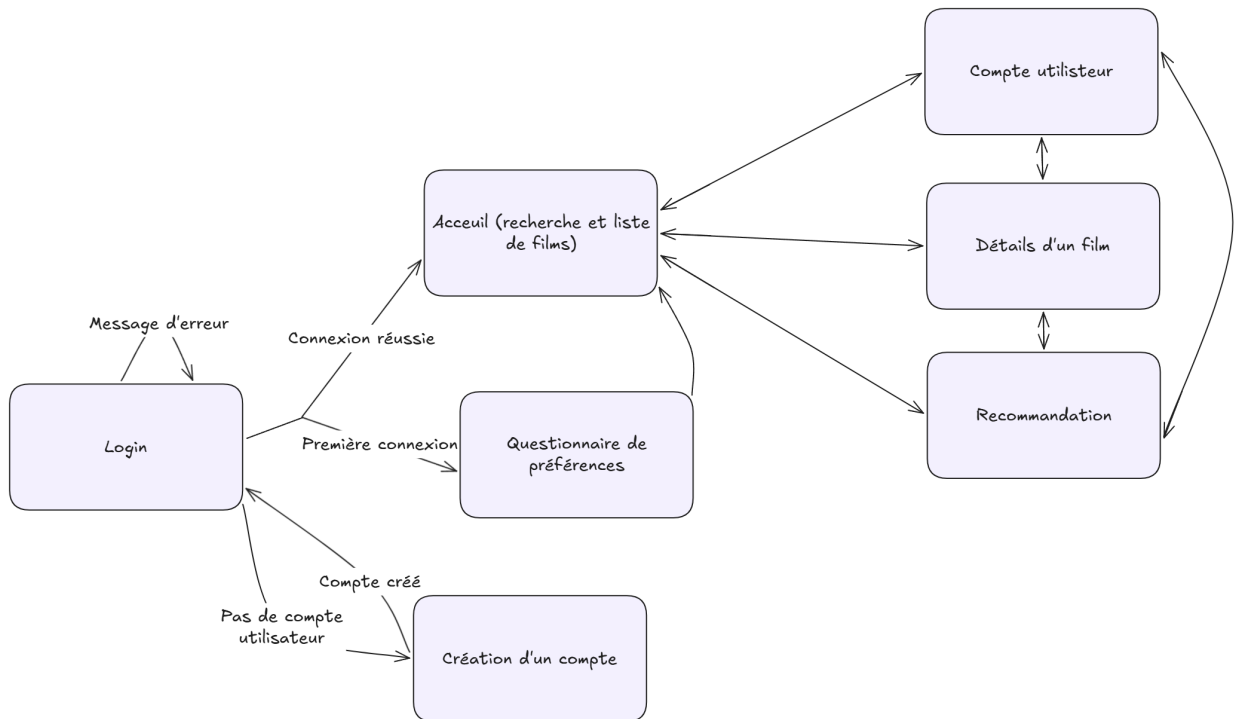


FIGURE 4.6 – Storyboard

Chapitre 5

Conception détaillée

5.1 Description des modules — Frontend

5.1.1 Interface utilisateur – application Vue.js

Description générale

Le module d'interface utilisateur constitue la couche de présentation de l'application.

Il permet aux utilisateurs d'interagir avec le système de recommandation de films à travers une interface web développée avec Vue.js et déployée via AWS Amplify Hosting.

Ce module couvre les interactions principales avec l'application :

- Création de compte utilisateur (FR-03).
- Authentification et connexion (FR-04).
- Formulaire de première connexion (FR-05).
- Consultation des films (FR-08).
- recherche de films (FR-09).
- Notation des films (FR-07).
- Affichage des recommandations personnalisées (FR-06).

L'interface communique avec les services backend via AWS AppSync (GraphQL) pour l'accès données et Amazon API Gateway (rest) pour l'appel du moteur de recommandation.

Composants utilisés / utilisateurs

Utilise :

- AWS Amplify Auth pour l'authentification via Amazon Cognito.
- AWS AppSync (GraphQL) pour accéder aux données stockées dans DynamoDB.
- Amazon API Gateway pour appeler la fonction Lambda de recommandation.
- Vue Router pour la gestion de la navigation entre les pages.
- Tailwind CSS pour la mise en forme de l'interface utilisateur.
- FontAwesome pour l'utilisation d'icônes dans l'interface.

Utilisé par :

- Les utilisateurs de l'application via un navigateur web.

Interface avec le backend

La communication entre le frontend et le backend s'effectue via des requêtes HTTPS sécurisées. Deux types d'API sont utilisées :

API GraphQL - AWS AppSync

Les principales opérations utilisées :

- Récupération des films.
- Recherche de film.
- Gestion des préférences utilisateur.
- Sauvegarde des notes.
- Récupération du profil utilisateur.

Exemples d'opérations :

- `getMovie(movieId)`
- `listMovies(limit,nextToken)`
- `searchMovies(keyword)`
- `saveRating(userId)`

API Rest – Amazon API Gateway

Un endpoint REST est utilisé pour récupérer les recommandations personnalisées.

Endpoint : `GET /recommendations`

Entrée : Token JWT cognito transmis dans le header HTTP.

Sortie : Liste d'au moins 6 films recommandés.

5.1.2 Module de navigation - Vue Router

Description générale

La navigation entre les pages de l'application est gérée par la bibliothèque Vue Router.

Ce module permet de définir les différentes routes correspondant aux vues de l'interface et d'assurer une navigation fluide sans rechargement complet de la page.

Certaines routes sont protégées et nécessitent que l'utilisateur soit authentifié avant d'y accéder.

Routes principales :

Route	Composant Vue	Description
<code>/login</code>	<code>LoginPage.vue</code>	Page de connexion
<code>/register</code>	<code>RegisterPage.vue</code>	Page de création de compte
<code>/profile</code>	<code>ProfilePage.vue</code>	Page de profil et informations sur les données enregistrées du formulaire de première connexion
<code>/movie</code>	<code>MoviePage.vue</code>	Page d'affichage de la liste des films et de recherche
<code>/movie/:id</code>	<code>MovieDetailsPage.vue</code>	Affichage des détails d'un film
<code>/form</code>	<code>FormPage.vue</code>	Page du formulaire de première connexion

TABLE 5.1 – Les routes principales de l'application Vue.js

5.1.3 Module Composants Vue

Description générale

L'interface est composée de plusieurs composants Vue réutilisables permettant de structurer l'affichage et de faciliter la maintenance du code.

Ces composants sont utilisés dans les différentes vues de l'application.

Composants principaux :

Composant	Rôle
Navbar.vue	Barre de navigation
MovieCard.vue	Affichage d'un film
RatingComponent.vue	Système de notation
SearchBar.vue	Barre de recherche
RecommendationList.vue	Affichage des films recommandés

TABLE 5.2 – Les composants principaux de l'interface utilisateur

Ces composants permettent de séparer la logique de présentation et d'assurer une meilleure réutilisation du code.

5.1.4 Module Communication API

Le module communication API assure l'échange de données entre l'interface web et les services backend de l'application.

Il centralise les appels aux différents services AWS afin de séparer la logique de communication réseau de la logique d'interface utilisateur.

Ce module utilise le cadre que propose AWS Amplify afin de faciliter l'intégration entre l'application frontend développée en Vue.js et TypeScript et les services backend tel que AWS AppSync et Amazon API Gateway.

La centralisation des appels API permet d'améliorer la maintenabilité du code et de simplifier l'utilisation des données dans les différents composants de l'interface utilisateur.

Composants utilisés / utilisateurs

Utilise :

- AWS Amplify API pour la communication avec les services backend.
- AWS AppSync (GraphQL) accéder aux données stockées dans DynamoDB.
- Amazon API Gateway (REST) pour l'appel du moteur de recommandation.
- TypeScript pour le typage des données échangées entre le frontend et le backend.

Utilisé par :

- Les différents composants Vue de l'application.
- Les vues principales de l'interface utilisateur (films, recherche, recommandations).

Services principaux :

Service	Rôle
auth.ts	gestion de l'authentification avec Amazon Cognito
api.ts	communication avec l'API GraphQL AWS AppSync
recommendation.ts	appel de l'endpoint REST /recommendation

TABLE 5.3 – Les services utilisés pour la communication avec les API

Ces services permettent d'isoler les appels aux API et de faciliter leur utilisation dans les composants Vue.

5.2 Description des modules — Backend

5.2.1 Module Authentification — AWS Cognito (Amplify Auth)

Description générale

Ce module couvre la création de compte (FR-03) et l'authentification (FR-04). Il est entièrement géré par AWS Cognito via Amplify Auth : l'équipe configure le service dans `amplify/auth/resource.ts` sans écrire de logique d'authentification. Cognito génère les tokens JWT utilisés par AppSync pour identifier l'utilisateur sur chaque requête GraphQL.

Composants utilisés / utilisateurs

Utilise : service AWS Cognito (User Pool), Amplify Auth (configuration déclarative TypeScript).

Utilisé par :

- AWS AppSync (vérifie le token JWT pour les requêtes GraphQL)
- Amazon API Gateway (vérifie le token JWT pour `GET /recommendations`)
- Module Lambda de recommandation (récupère le `userId` depuis le contexte de la requête)

Attributs Cognito générés automatiquement

Attribut	Rôle
sub (userId)	Identifiant unique de l'utilisateur généré par Cognito. Il sert de clé principale dans toutes les collections DynamoDB du projet.
email	Adresse email de l'utilisateur utilisée comme identifiant de connexion (<code>loginWith: email</code>).
email_verified	Booléen indiquant si l'adresse email a été confirmée par l'utilisateur après réception du code de vérification.
custom :username	Nom d'utilisateur affiché dans l'interface de l'application (attribut personnalisé optionnel).

TABLE 5.4 – Attributs utilisateur générés par Amazon Cognito

Flux d'inscription :

1. L'utilisateur saisit son email et son mot de passe dans l'application Vue.js
2. Amplify Auth appelle le service AWS Cognito
3. Cognito envoie un email contenant un code de vérification
4. L'utilisateur saisit le code reçu
5. Le compte est activé

5.2.2 Module Schéma de données — Amazon DynamoDB

Description générale

DynamoDB est la base de données NoSQL du projet, configurée via Amplify Data. Elle stocke toutes les données métier. Chaque collection utilise le *sub* Cognito (userId) comme clé de partition pour les données personnelles. DynamoDB n'est pas une base de données relationnel : il n'y a pas de jointures, les données sont conçues pour répondre efficacement aux accès les plus fréquents.

Collection Movie

Contient les films importés depuis *The Movies Dataset*. Cette collection est en lecture seule depuis l'application et est importée une seule fois lors de l'initialisation du système.

Attribut (clé DynamoDB)	Type	Description
movieId (PK)	String	Identifiant unique du film (tmdbId). Clé de partition.
title	String	Titre du film.
overview	String	Résumé du film.
genres	List	Liste des genres (ex. : ["Action", "Drama"]).
keywords	List	Liste des mots-clés de film. Utilisée par l'algorithme TF-IDF.
releaseDate	String	Date de sortie du film (format YYYY-MM-DD).
voteAverage	Number	Note moyenne issue du dataset (0 à 10).
voteCount	Number	Nombre de votes présents dans le dataset.
director	String	Nom du réalisateur extrait de credits.csv.
posterPath	String	Chemin de l'affiche (URL base TMDB).

TABLE 5.5 – Schéma de la collection Movie

Collection Rating

Stocke les notes attribuées par les utilisateurs aux films (FR-07). Une clé composite (userId + movieId) est utilisée afin de garantir l'unicité de chaque notation.

Attribut (clé DynamoDB)	Type	Description
userId (PK)	String	Identifiant Cognito (sub). Clé de partition.
movieId (SK)	String	Identifiant du film. Clé de tri permettant de récupérer toutes les notes d'un utilisateur en une seule requête.
rating	Number	Note attribuée par l'utilisateur (0.5 à 5.0 par pas de 0.5).
isInitial	Boolean	True si la note provient du questionnaire initial (FR-05). Permet la remise à zéro sélective.
createdAt	String	Date de création de la note (format ISO 8601).
updatedAt	String	Date de dernière modification.

TABLE 5.6 – Schéma de la collection Rating

Collection Preference

Stocke les genres favoris sélectionnés lors du questionnaire initial (FR-05). Ces données servent d'amorce pour l'algorithme de recommandation lorsque l'utilisateur n'a pas encore attribué de notes.

Attribut (clé DynamoDB)	Type	Description
userId (PK)	String	Identifiant Cognito (sub). Clé de partition.
genres	List	Liste des genres sélectionnés (ex. : ["Action", "Comedy"]).
hasCompleted	Boolean	True si le questionnaire initial a été complété. Utilisé par le frontend pour contrôler l'affichage du questionnaire.
createdAt	String	Date de création de l'enregistrement.

TABLE 5.7 – Schéma de la collection Preference

Collection UserProfile

Stocke les informations complémentaires du profil utilisateur qui ne sont pas gérées directement par Amazon Cognito.

Attribut (clé DynamoDB)	Type	Description
userId (PK)	String	Identifiant Cognito (sub). Clé de partition.
username	String	Nom d'utilisateur affiché dans l'interface de l'application.
createdAt	String	Date de création du profil utilisateur.

TABLE 5.8 – Schéma de la collection UserProfile

5.2.3 Module Couche API — AppSync (GraphQL) + API Gateway (REST)

Description générale

La couche API combine deux services complémentaires. AWS AppSync expose une API GraphQL qui gère toutes les opérations CRUD courantes : films, notes, préférences et profil utilisateur. AppSync génère automatiquement les résolveurs DynamoDB via Amplify. Amazon API Gateway expose un endpoint REST (`GET /recommendations`) qui déclenche la fonction Lambda Python de recommandation. Les deux services authentifient chaque requête via le token JWT Cognito.

Composants utilisés / utilisateurs

Utilise : collections DynamoDB (Movie, Rating, Preference, UserProfile), fonction Lambda Python (Module 4.4 — via API Gateway uniquement), Amazon Cognito (vérification JWT).

Utilisé par : frontend Vue.js via le client Amplify généré automatiquement.

Règles d'autorisation

La règle `allow.owner()` dans le schéma Amplify garantit qu'un utilisateur ne peut lire et modifier que ses propres données. Les films sont accessibles en lecture à tous les utilisateurs authentifiés.

Opérations exposées — AppSync (GraphQL) et API Gateway (REST)

Opération	Service	Type	Entrée	Retour
createUserProfile	AppSync (GraphQL)	Mutation	username	Profil créé
savePreferences	AppSync (GraphQL)	Mutation	userId, genres[]	Préférences enregistrées
resetPreferences	AppSync (GraphQL)	Mutation	userId	Préférences supprimées
saveRating	AppSync (GraphQL)	Mutation	userId, movieId, rating (0.5–5.0)	Note enregistrée
deleteRating	AppSync (GraphQL)	Mutation	userId, movieId	Note supprimée
getUserProfile	AppSync (GraphQL)	Query	userId	Profil utilisateur
getPreferences	AppSync (GraphQL)	Query	userId	Genres préférés, has-Completed
getMovie	AppSync (GraphQL)	Query	movieId	Détails du film
listMovies	AppSync (GraphQL)	Query	limit, nextToken	Liste paginée de films
searchMovies	AppSync (GraphQL)	Query	keyword	Films correspondants
getUserRatings	AppSync (GraphQL)	Query	userId	Notes de l'utilisateur
GET /recommendations	API Gateway (REST)	REST GET	JWT (header)	≥ 6 films recommandés

TABLE 5.9 – Opérations exposées par la couche API

5.2.4 Module Lambda Python — Algorithme de recommandation

Description générale

C'est le seul module entièrement codé en Python par l'équipe. La Lambda est déclenchée par Amazon API Gateway lorsque le frontend appelle GET /recommendations. Elle récupère les données de l'utilisateur depuis DynamoDB, calcule la similarité cosinus entre son profil et tous les films, et retourne au minimum 6 films non encore notés. Elle couvre FR-01 et FR-06.

Composants utilisés/utilisateurs

- Utilise : DynamoDB (collections Rating, Preference, Movie), bibliothèques Python : boto3, numpy, scikit-learn.
- Utilisé par : Amazon API Gateway (déclenche la Lambda via endpoint REST GET /recommendations).

Structure des fichiers

Fichier	Rôle
<code>handler.py</code>	Point d'entrée de la fonction Lambda. Contient la fonction <code>lambda_handler(event, context)</code> qui orchestre les appels aux autres modules.
<code>dynamo_client.py</code>	Fonctions d'accès à DynamoDB : lecture des notes utilisateur, préférences et films.
<code>recommender.py</code>	Implémentation de l'algorithme de recommandation : construction du profil utilisateur, calcul de similarité cosinus, filtrage et tri des résultats.
<code>requirements.txt</code>	Dépendances Python utilisées par la Lambda : <code>numpy</code> , <code>scikit-learn</code> , <code>boto3</code> .

TABLE 5.10 – Structure des fichiers du module de recommandation

Fonctions — dynamo_client.py

Fonction Python	Paramètres	Retour	Rôle
<code>get_user_ratings()</code>	<code>user_id : str</code>	<code>list[dict]</code>	Requête DynamoDB sur la collection Rating (PK=userId). Retourne <code>[movieId, rating, ...]</code> .
<code>get_user_preferences()</code>	<code>user_id : str</code>	<code>list[str]</code>	Requête DynamoDB sur la collection Preference. Retourne la liste des genres favoris.
<code>get_all_movies()</code>	—	<code>list[dict]</code>	Scan de la collection Movie. Retourne tous les films avec leur vecteur TF-IDF pré-calculé.
<code>get Rated movie ids()</code>	<code>user_id : str</code>	<code>list[str]</code>	Retourne uniquement les <code>movieId</code> déjà notés afin de les exclure des recommandations.

TABLE 5.11 – Fonctions d'accès à DynamoDB

Fonction Python	Paramètres	Retour	Rôle
<code>build_tfidf_matrix()</code>	<code>movies : list</code>	<code>matrix, vectorizer, index_map</code>	Construit la matrice TF-IDF à partir des genres et des mots-clés. Appelée une seule fois lors du pré-traitement du dataset.
<code>build_user_profile()</code>	<code>ratings : list, movies_map : dict</code>	<code>np.ndarray</code> <code>None</code>	Construit le profil utilisateur comme somme pondérée des vecteurs TF-IDF des films notés ≥ 3.5 . Retourne <code>None</code> si aucune note valide.
<code>get_genre_profile()</code>	<code>genres : list, movies_map : dict</code>	<code>np.ndarray</code> <code>None</code>	Pour une première connexion basée sur les genres favoris remplis dans le formulaire. Retourne <code>None</code> si aucun film correspondant.
<code>compute_scores()</code>	<code>user_vec, movies_map : dict</code>	<code>list[dict]</code>	Calcule la similarité cosinus entre le profil utilisateur et chaque film non encore vu. Retourne <code>[film, score, ...]</code> .
<code>get_recommendations()</code>	<code>user_id : str, n : int = 10</code>	<code>list[dict]</code>	Fonction principale qui orchestre toutes les étapes de l'algorithme de recommandation (voir pseudo-code section 4.4.9).

TABLE 5.12 – Fonctions de l'algorithme de recommandation

Pseudo-code — `get_recommendations()`

Rôle : Fonction orchestratrice appelée par `lambda_handler()`. Elle coordonne toutes les étapes : récupération des données DynamoDB, choix de la stratégie de profil (notes ou genres), calcul des scores de similarité cosinus, exclusion des films déjà vus, tri et retour du résultat final. C'est le point d'entrée de toute la logique de recommandation.

Algorithm 1: `get_recommendations`

Input: `user_id`, $n = 10$

Output: Liste de n films recommandés

// 1. Récupérer les données de l'utilisateur

`ratings` $\leftarrow$ `get_user_ratings(user_id)`

`preferences` $\leftarrow$ `get_user_preferences(user_id)`

`movies` $\leftarrow$ `get_all_movies()`

// 2. Construire `movies_map` pour accès rapide

`movies_map` $\leftarrow$ $\{film.movieId : film \mid film \in movies\}$

// 3. Construire le profil utilisateur

if `ratings` n'est pas vide **then**

`user_profile` $\leftarrow$ `build_user_profile(ratings, movies_map)`

else

`user_profile` $\leftarrow$ `get_genre_profile(preferences.genres, movies_map)`

// 4. Calculer les scores de similarité

`scores` $\leftarrow$ `compute_scores(user_profile, movies)`

// 5. Exclure les films déjà notés

`rated_ids` $\leftarrow$ `get_rated_movie_ids(user_id)`

`scores` $\leftarrow$ `scores` sans les films dont `movieId` $\in$ `rated_ids`

// 6. Trier et retourner les n meilleurs films

trier `scores` par ordre décroissant

return les n premiers éléments de `scores`

5.2.5 Module Amazon S3 — Stockage des fichiers

Description générale

Amazon S3 est utilisé comme espace de stockage de fichiers statiques et de données volumineuses qui ne sont pas adaptées à DynamoDB. Il stocke le dataset *The Movies Dataset* importé depuis Kaggle, les posters des films ainsi que les éventuels artefacts du modèle TF-IDF (vecteurs pré-calculés).

Ce module ne nécessite pas de code spécifique écrit par l'équipe : les opérations de lecture et d'écriture sont réalisées via les SDK AWS ou directement via la console AWS.

Contenu des buckets S3

- **dataset/** — contient les fichiers CSV du dataset *The Movies Dataset* : `movies_metadata.csv`, `credits.csv`, `keywords.csv`, `ratings_small.csv`.
- **posters/** — contient les images des affiches de films (format JPG ou PNG), récupérées via l'API TMDb ou extraites du dataset.
- **artifacts/** — contient les vecteurs TF-IDF pré-calculés. Ces fichiers sont optionnels si les vecteurs sont stockés directement dans DynamoDB ; ils peuvent être placés dans S3 pour éviter des limitations de taille.

Composants utilisés / utilisateurs

Utilisé par :

- Module Lambda Python de recommandation (lecture des artefacts TF-IDF si ceux-ci sont stockés dans S3)
- Script de pré-traitement du dataset (écriture du dataset transformé)
- Application frontend Vue.js (affichage des posters via une URL publique S3)

Chapitre 6

Annexes

A.1 Les fonctionnalités obligatoires du produit :

FR-01 Système de recommandation basique.

FR-02 Interface web fonctionnelle et interactive.

FR-03 Création d'un compte utilisateur.

FR-04 Authentification et connexion de l'utilisateur.

FR-05 Choix des préférences utilisateur, pour amorcer le moteur de recommandation.

FR-06 Génération d'une liste de films recommandés.

FR-07 Notation des films par l'utilisateur.

FR-08 Affichage des informations d'un film.

FR-09 Barre de recherche.

Glossaire

- API (Application Programming Interface) : Interface permettant à deux logiciels de communiquer entre eux. Dans ce projet, AppSync et API Gateway exposent des APIs que Vue.js appelle pour accéder aux données.
- API Gateway : Service AWS qui crée et gère des endpoints REST. Utilisé pour exposer l'endpoint GET /recommendations qui déclenche la Lambda Python.
- AppSync : Service AWS qui expose une API GraphQL. Gère automatiquement les opérations CRUD sur DynamoDB à partir du schéma défini dans resource.ts.
- Architecture : L'architecture d'un système informatique désigne l'organisation générale de ses composants logiciels et matériels ainsi que les interactions entre eux.
- Authentication : Processus permettant de vérifier l'identité d'un utilisateur, par exemple à l'aide d'un identifiant et d'un mot de passe.
- Authorization : Processus permettant de déterminer les actions ou ressources auxquelles un utilisateur authentifié peut accéder.
- AWS (Amazon Web Services) : Plateforme de services cloud d'Amazon. Ce projet utilise plusieurs services AWS : Amplify, Cognito, DynamoDB, AppSync, API Gateway, Lambda et S3.
- AWS Amplify : Framework AWS qui simplifie le développement d'applications web et mobiles. Il génère automatiquement l'infrastructure backend à partir d'un schéma TypeScript.
- Backend : Le backend est la partie invisible, qui gère le fonctionnement interne du produit (par exemple : les données).
- Base de données : Une base de données est un ensemble d'informations organisé de manière à être facilement accessible, géré et mis à jour.
- boto3 : Bibliothèque Python officielle d'AWS permettant d'interagir avec les services AWS depuis du code Python. Utilisée par la Lambda pour lire les collections DynamoDB.
- Cognito : Service AWS gérant l'authentification et les comptes utilisateurs. Il gère l'inscription, la connexion et génère les tokens JWT.
- CRUD : Acronyme des quatre opérations de base sur une base de données : Create (créer), Read (lire), Update (modifier), Delete (supprimer).
- Diagramme de séquences : Un diagramme de séquences est un type de diagramme UML qui représente l'ordre chronologique des interactions entre différents composants d'un système. Il montre comment les objets ou services échangent des messages au cours d'un scénario spécifique.
- Diagramme UML : Un diagramme UML (Unified Modeling Language) est une représentation graphique utilisée pour modéliser et documenter la structure ou le comportement d'un système logiciel.
- DynamoDB : Base de données NoSQL d'AWS. Stocke les données du projet : films, notes, préférences et profils utilisateurs sous forme de collections.

- Endpoint : URL précise exposée par une API pour recevoir des requêtes. Exemple : GET /recommendations est un endpoint d'API Gateway.
- Framework : Un framework est une structure prête à être utilisée pour développer un logiciel, il contient des outils et des bibliothèques.
- Frontend : Le frontend est la partie visible du site web, c'est ce avec quoi l'utilisateur interagit directement.
- GraphQL : Langage de requête pour les APIs permettant au client de demander exactement les champs dont il a besoin. Utilisé par AppSync dans ce projet.
- Interface web : Une interface web est une interface à laquelle on accède et qui s'affiche grâce à un navigateur web, qui permet un dialogue entre l'utilisateur et le système.
- JWT token : Un JWT (JSON Web Token) est un format de jeton utilisé pour transmettre des informations de manière sécurisée entre deux parties. Il est souvent utilisé dans les systèmes d'authentification pour représenter l'identité d'un utilisateur après sa connexion et permettre l'accès aux ressources protégées.
- Keywords : Mots-clés associés à chaque film dans le dataset (fichier keywords.csv). Concaténés avec les genres pour construire le vecteur TF-IDF de chaque film.
- Lambda : Service AWS permettant d'exécuter du code sans gérer de serveur. Dans ce projet, une Lambda Python calcule les recommandations TF-IDF.
- Mutation : Opération GraphQL qui modifie des données (créer, modifier, supprimer). Équivalent d'un POST/PUT/DELETE en REST.
- NoSQL : Type de base de données qui ne suit pas le modèle relationnel classique. Les données y sont stockées sous des formats flexibles comme clé-valeur ou document. DynamoDB est une base de données NoSQL.
- np.ndarray : Type de données NumPy représentant un tableau de nombres. Utilisé pour stocker les vecteurs TF-IDF des films et le profil mathématique de l'utilisateur.
- NumPy : Bibliothèque Python spécialisée dans les calculs mathématiques sur des tableaux de nombres. Utilisée pour calculer la similarité cosinus entre vecteurs.
- Query : Opération GraphQL qui lit des données sans les modifier. Équivalent d'un GET en REST.
- REST (Representational State Transfer) : Style d'architecture pour les APIs utilisant des URLs et des méthodes HTTP (GET, POST...). API Gateway expose une API REST dans ce projet.
- Résolveur : Code qui traduit une opération GraphQL en action concrète sur la base de données. Amplify génère automatiquement les résolveurs DynamoDB à partir du schéma.
- S3 (Simple Storage Service) : Service de stockage de fichiers AWS. Utilisé pour stocker le dataset The Movies Dataset, les posters de films et les artifacts TF-IDF.
- SDK (Software Development Kit) : Boîte à outils permettant d'utiliser un service depuis un langage de programmation. boto3 est le SDK Python d'AWS.
- Similarité cosinus : Mesure mathématique calculant la ressemblance entre deux vecteurs. Retourne un score entre 0 (aucune ressemblance) et 1 (identiques). Utilisée pour comparer le profil utilisateur avec les films.
- Stockage d'objet : Le stockage d'objets consiste à traiter chaque donnée comme un objet individuel, plutôt que comme un bloc ou un fichier faisant partie d'un système de fichiers hiérarchique.
- Système de recommandation : Un système de recommandation est un système qui permet de proposer des contenus ou des produits en fonction de ses préférences.

- TF-IDF (Term Frequency – Inverse Document Frequency) : Algorithme qui transforme du texte en vecteur numérique en pondérant l'importance de chaque mot. Utilisé pour représenter les films à partir de leurs genres et keywords.
- TfidfVectorizer : Classe Python de la bibliothèque scikit-learn qui applique l'algorithme TF-IDF sur du texte et retourne des vecteurs `np.ndarray`.
- Vue.js : Framework JavaScript utilisé pour le frontend de l'application. Il communique avec AppSync (GraphQL) et API Gateway (REST).

Références

- Rounak, Banik. « The Movies Dataset ». 2018,
<https://www.kaggle.com/datasets/rounakbanik/the-movies-dataset/data?select=credits.csv>.
- Fournier-S'niehotta, Raphaël. « Systèmes de recommandation »,
<https://cedric.cnam.fr/vertigo/cours/RCP217/docs/RCP217-RecSys1.pdf>
- « Amazon Cognito ». Amazon Web Services, Inc.,
<https://aws.amazon.com/fr/pm/cognito/>
- «Amazon DynamoDB». Amazon Web Services, Inc.,
<https://aws.amazon.com/fr/dynamodb/>
- «API graphiques sans serveur - AWS AppSync - AWS». Amazon Web Services, Inc.,
<https://aws.amazon.com/fr/appsync/>
- « Gestion des API - Amazon API Gateway - AWS ». Amazon Web Services, Inc.,
<https://aws.amazon.com/fr/api-gateway/>
- « Fonction sans serveur, FaaS sans serveur - AWS Lambda - AWS ». Amazon Web Services, Inc.,
<https://aws.amazon.com/fr/lambda/>
- « Amazon S3 – Stockage d’objets dans le cloud – AWS ». Amazon Web Services, Inc.,
<https://aws.amazon.com/fr/s3/>
- scikit-learn : machine learning in Python — scikit-learn 1.8.0 documentation.
<https://scikit-learn.org/stable/>
- pandas - Python Data Analysis Library.
<https://pandas.pydata.org/>
- Vue.Js.
<https://vuejs.org/>
- Vue Router | The Official Router for Vue.Js.
<https://router.vuejs.org/>
- Tailwind CSS - Rapidly Build Modern Websites without Ever Leaving Your HTML.
<https://tailwindcss.com/>
- Classic Icon Pack | Font Awesome.
<https://fontawesome.com>
- Handbook - The TypeScript Handbook.
<https://www.typescriptlang.org/docs/handbook/intro.html>
- OpenClassrooms.
<https://openclassrooms.com/fr/courses/8186936-gerez-un-projet-avec-la-methodologie-8303259-debutez-la-phase-de-conception-detaillee>

Index

- Amazon S3 26
- Api Gateway 20
- AppSync 20
- Architecture 9
- AWS Cognito 16
- Diagramme UML 10
- Domaines fonctionnels 8
- DynamoDB 17
- dynamo_client.py 23
- Lambda Python 22
- recommender.py 24
- Vue.js ??
- TypeScript ??
-